Hoofdstuk 2: Sorteeralgoritmes

Wat is Sorteren?

Sorteren = het ordenen van een lijst op basis van een vergelijkingscriterium (<, >, alfabetisch, chronologisch).

Sorteren wordt al bestudeerd sinds de jaren '50. Recente algoritmes zoals **Timsort** (2002) zijn nog steeds populair.

Complexiteit & Big O Notatie

Big O dient om algoritmes onderling te vergelijken op:

- Tijdsefficiëntie
- Geheugengebruik (space complexity)

Notatie	Betekenis
O(1)	Wordt niet trager bij meer input
O(n)	Dubbel zo traag bij dubbel zoveel input
$O(n^2)$	4x zo traag bij dubbel zoveel input
O(n log n)	Efficiënt voor grotere inputs

Bubble Sort

Bubble sort doorloopt de lijst en wisselt buren als ze in verkeerde volgorde staan. Herhaal tot de lijst gesorteerd is.

```
def bubble_sort(sequence):
    n = len(sequence)
    for i in range(n-1):
        for j in range(n-i-1):
            if sequence[j] > sequence[j+1]:
                sequence[j], sequence[j+1] = sequence[j+1], sequence[j]
    return sequence
```

Complexiteit: $O(n^2)$ in het slechtste geval.

Merge Sort

Merge sort gebruikt een **verdeel-en-heers** strategie (recursief).

```
def merge_sort(sequence):
    size = len(sequence)
    if size > 1:
        middle = size // 2
        left_arr = sequence[:middle]
        right_arr = sequence[middle:]

        merge_sort(left_arr)
        merge_sort(right_arr)

        p = q = r = 0
        while p < len(left_arr) and q < len(right_arr):
            if left_arr[p] < right_arr[q]:
                sequence[r] = left_arr[p]
                p += 1
            else:
                sequence[r] = right_arr[q]
                q += 1
            r += 1

        while p < len(left_arr):
            sequence[r] = left_arr[p]
            p += 1
            r += 1

        while q < len(right_arr):
            sequence[r] = right_arr[q]
```

Bubble vs Merge Sort Vergelijking

```
import random

# Test met een timer decorator
from time import time

def timer_func(func):
    def wrap_func(*args, **kwargs):
        t1 = time()
        result = func(*args, **kwargs)
        t2 = time()
        return result, t2 - t1
    return wrap_func

r = 100
n = 4000
average_bubble = 0
for i in range(r):
    sequence = random.sample(range(n), n)
    result = bubble_sort(sequence)
    average_bubble += result[1] / r
```

Andere Sorteeralgoritmes

- **Insertion sort**: efficiënt voor kleine lijsten
- **Quicksort**: snel, O(n log n) gemiddeld
- **Tree sort**: specifiek voor dynamische lijsten